Code of the Day

- What does the following code do? i.e., what can you tell from c

```
int c;
int v;
for (c=0; v; v>>=1) {
    c += (v & 1);
}
```

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Cache

02/07/2024

Before we get started…

- **Quick overview, things discussed last time**
  - Timing the execution of a program
  - Memory aspects
    - SRAM (fast but costly), DRAM (not so fast but cheap)
    - The memory hierarchy (SRAM-based cache + DRAM-based main memory)

- **Purpose of today's lecture:**
  - Understand the importance of locality
  - Understand the design of cache

- **Miscellaneous**
  - Programming Assignment #2 released today – **Due 2/17 23:59 PM** via Canvas

Regarding Your Programming Assignment #1…

- You received “0” for the following mistakes
  - Forgot to specify the GitHub Link to your source code: https://github.com/YourName/repo759/HW01
  - Forgot to follow the naming rule – e.g., should be “repo759/HW01” not “repo759/HW1”
    - We are doing auto-grading using the script! It’s important to ensure your spelling is correct!
  - Forgot to invite TA to your GitHub account as collaborators (tutorial here)
    - GitHub account names of TA are available on the Canvas home page

- Since this is your first programming assignment, we grace you a chance to regrade
  - You need to stop by TA’s office hour to regrade your assignment if you made the above mistakes

- All regrading requests must be done within 2 weeks after we release the score

Case Study: Adding the Entries in a 2D Matrix

- Do a case study to show how “locality” and “caches” impact the program performance
- Related to your assignment #2 – you’ll handle this 2D matrix quite a lot

Example: Add All the Entries in a 2D Matrix

- Two equivalent ways to add the entries in a matrix

$$\begin{pmatrix} 2 & -3 & 0 \\ -1 & 1 & 7 \end{pmatrix}$$

- Row-wise:

$$sumElements = \overbrace{(2 \ -3+0)}^{\text{First row}} + \overbrace{(-1+1+7)}^{\text{Second row}} = 6$$

- Column-wise:

$$sumElements = \overbrace{(2 \ -1)}^{\text{First column}} + \overbrace{(-3+1)}^{\text{Second column}} + \overbrace{(0+7)}^{\text{Third column}} = 6$$

Storing the 2D Matrix: Linearized format vs. 2D format

- Storing a matrix: linearized format or 2D format

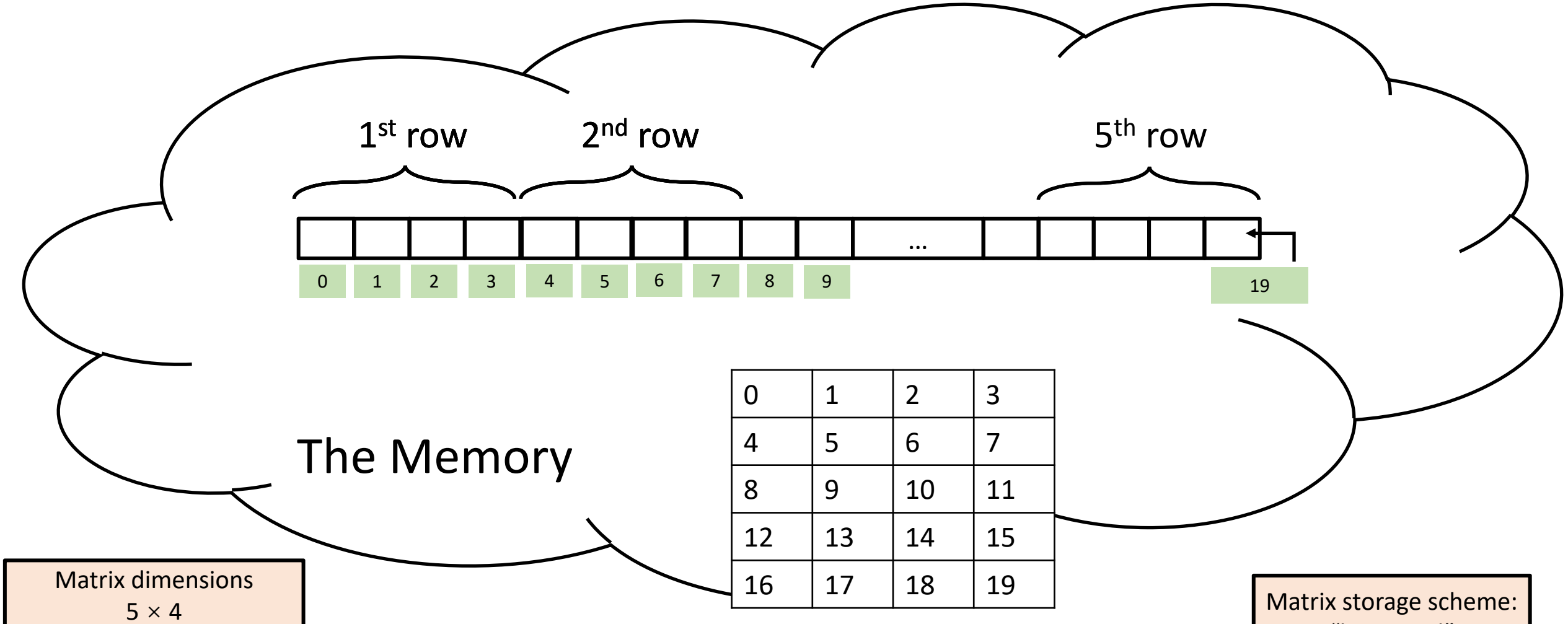
$$\begin{pmatrix} 2 & -3 & 0 \\ -1 & 1 & 7 \end{pmatrix}$$

- Linearized format – using one long contiguous array of 6 entries.
  - int array[6] = {2, -3, 0, -1, 1, 7}

- The 2D format – using two arrays:
  - Ex: int array1[3] = {2, -3, 0}, array2[3] = {-1, 1, 7};
  - Each has 3 entries
  - Each of the two arrays stored somewhere in the memory (they might be far from each other)

Matrix Stored in Linearized Format (row major)

Boxes of this color used to show sequence of entries read for row-wise access of elements



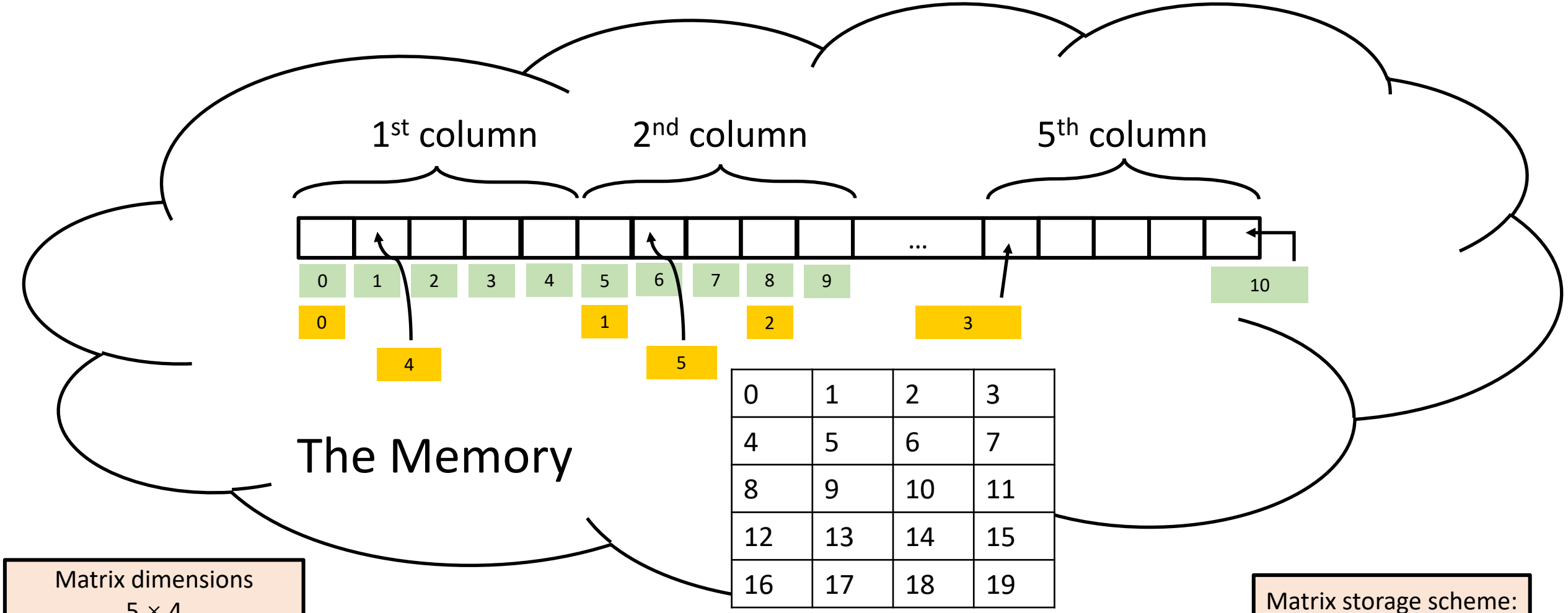
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15
16	17	18	19

Matrix dimensions
5 × 4
20 elements

Matrix storage scheme:
“linearized”

Matrix Stored in Linearized Format (column major)

Boxes of this color used to show sequence of entries read for row-wise access of elements
Boxes of this color used to show sequence of entries read for column-wise access of elements



0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15
16	17	18	19

Matrix dimensions
5 × 4
20 elements

Matrix storage scheme:
“linearized”

Mapping between a 2D Matrix and its Linearized 1D Indices (Row Major)

- Index mapping in row-major (row-by-row) storage: a[i][j] → i*Columns + j

Columns

Rows

2D Array	0	1	2
0	a[0][0] 0	a[0][1] 1	a[0][2] 2
1	a[1][0] 3	a[1][1] 4	a[1][2] 5
2	a[2][0] 6	a[2][1] 7	a[2][2] 8

Mapping between a 2D Matrix and its Linearized 1D Indices (Column Major)

- Index mapping in column-major (column-by-column) storage: a[i][j] → j*Rows + i

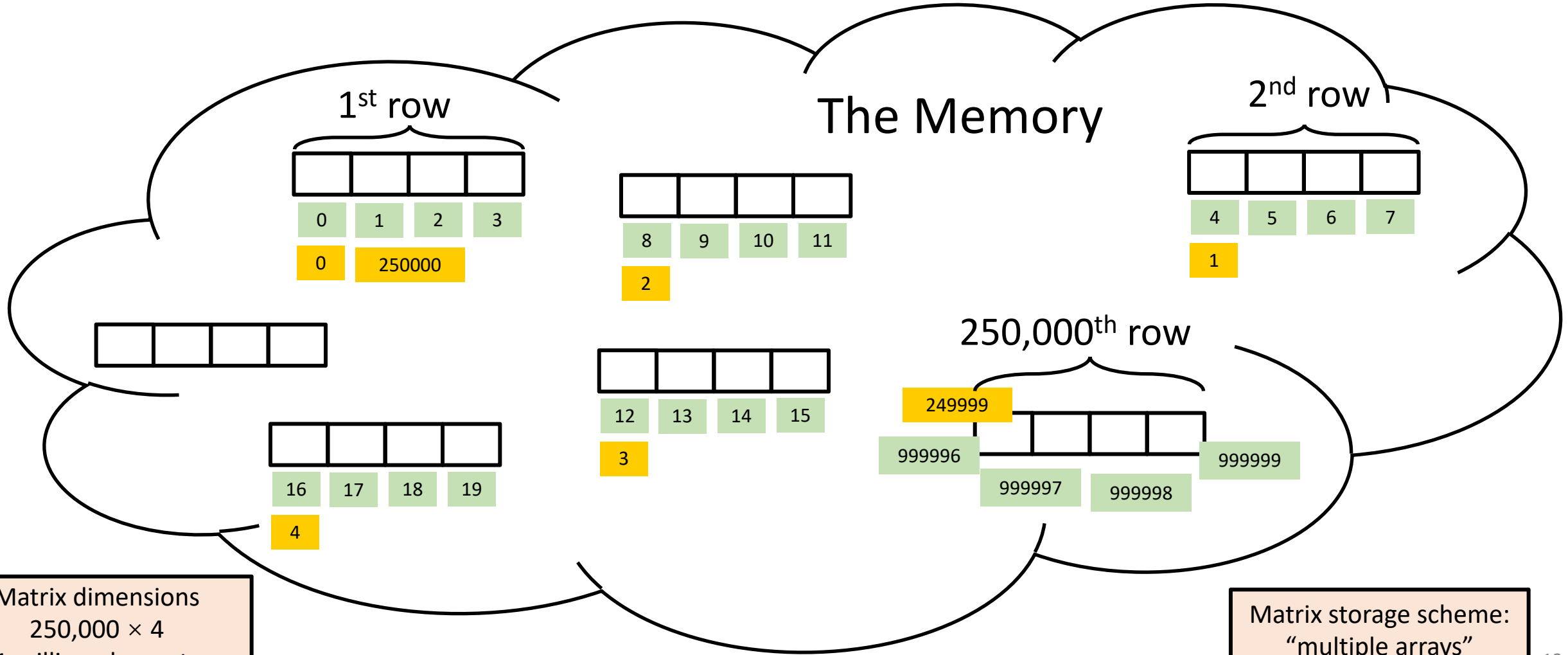
Columns

Rows

2D Array	0	1	2
0	a[0][0] 0	a[0][1] 3	a[0][2] 6
1	a[1][0] 1	a[1][1] 4	a[1][2] 7
2	a[2][0] 2	a[2][1] 5	a[2][2] 8

Matrix Stored in Multiple Arrays

Boxes of this color used to show sequence of entries read for row-wise access of elements
Boxes of this color used to show sequence of entries read for column-wise access of elements

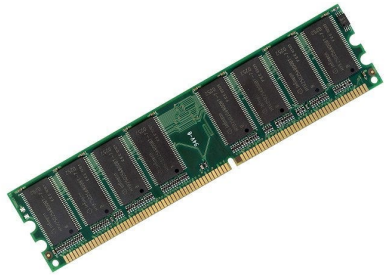


Matrix dimensions
250,000 × 4
1 million elements

Matrix storage scheme:
"multiple arrays"

Row Major vs Column Major

Features	Row major (C, C++, Python)	Column Major (Matlab, Fortran)
Memory layout	Stores rows **contiguously** before moving to the next row	Stores columns **contiguously** before moving to the next column
Access pattern	Best for row-wise traversal (left to right)	Best for column-wise traversal (top to bottom)
Pointer arithmetic	arr[i][j] = base + (i * cols + j)	arr[j][i] base + (j * rows + i)
Cache efficiency	Better for row-wise loops (good spatial locality)	Better for column-wise loops, but bad for row-wise loops due to cache misses
Loop	Faster for iterating through rows (nature in C, C++, Python)	Faster for iterating through columns (nature in Fortran)



Keep in mind your memory (hardware) is ultimately just a linear storage to store your data!

Side Trip to Dynamic Memory Allocation in C

- Dynamic memory allocation: `malloc`
  - A process of allocating memory at runtime
  - Useful when the data size is unknown in advance or when managing large datasets that vary in size

```
int *ids;  //id arrays
int num_of_ids = 40;
ids = (int*)malloc( sizeof(int)*num_of_ids );
//… do your work here…
free(ids);
```

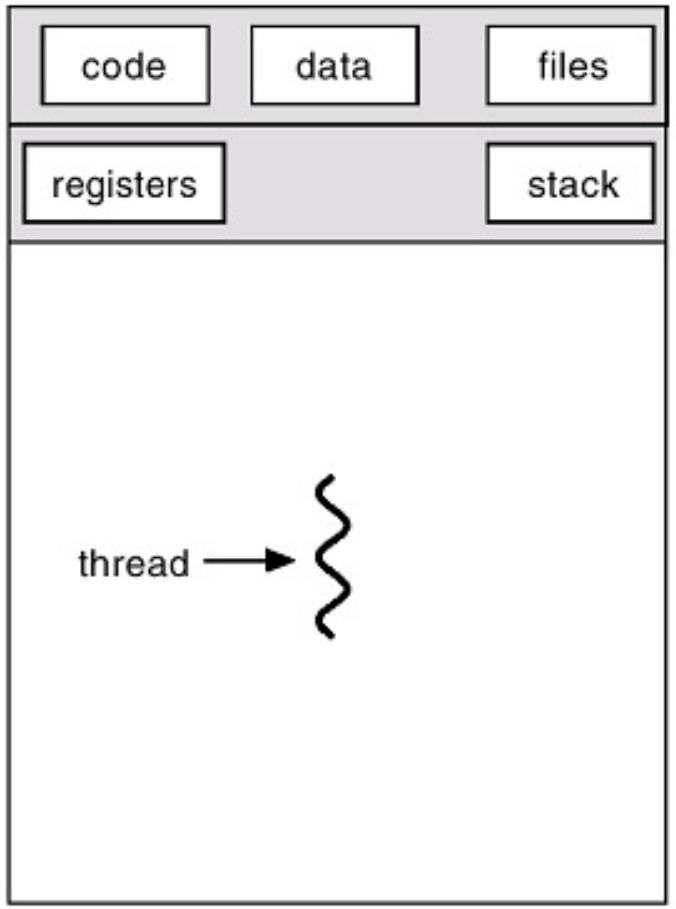
- *Three basic functions* come into play in conjunction with dynamic memory allocation
  - IMPORTANT: `sizeof()` is a compile-time operator that returns the size in terms of bytes of the given type
    - Ex: `sizeof(int)` is 4
    - Ex: `sizeof(std::string)` is 32

```
void *malloc(size_t number_of_bytes);
// allocates dynamic memory

size_t sizeof(someType);
// returns the number of bytes of someType

void free(void * p)
// releases dynamic memory allocated
```

Quiz: Why do we need `malloc`?

- What happened to the following code?

```
int array1[100];

int array2[1,000,000,000];
```



single-threaded

Quiz: Difference between `new` and `malloc`

- What are the differences between `new` and `malloc`?

```
int* ptr1 = new int;

int* ptr2 = (int*)malloc(sizeof(int));
```

Quiz: Difference between `delete` and `free`

- What are the differences between `delete` and `free`?

```
int* ptr1 = new int;

int* ptr2 = (int*)malloc(sizeof(int));

delete ptr1;

free(ptr2)
```

Store a 2D matrix in a Linearized 1D Array

```c
#include <stdio.h>
#include <stdlib.h>

struct squashedMatrix {
  /// 2D matrix stored row-wise in a one dimensional array
  unsigned int height; /// number of rows in matrix
  unsigned int width;  /// number of columns in matrix
  double *pMatVals;
};

int main() {
  struct squashedMatrix mat1D;

  mat1D.height = 1000; // perhaps you set this based on user input
  mat1D.width = 2000;  // perhaps you set this based on user input
  mat1D.pMatVals = (double*)malloc(mat1D.height*mat1D.width*sizeof(double));

  // initialize the matrix (1D index of A[i][j] = i*Columns + j)
  unsigned int i, j;
  for (i = 0; i < mat1D.height; i++)
    for (j = 0; j < mat1D.width; j++)
      mat1D.pMatVals[i * mat1D.width + j] = 1. / (i + j + 1.);

  // Compute sum of elements
  double sum = 0.;
  for (i = 0; i < mat1D.height; i++)
    for (j = 0; j < mat1D.width; j++)
      sum += mat1D.pMatVals[i * mat1D.width + j];

  printf("The sum of the elements is: %f\n", sum);

  // Done with the matrix; free the mem
  free(mat1D.pMatVals);

  return 0;
}
```

- One long array stores all entries in the matrix
  - Ex: a 1000×2000 matrix will need an array of 2000000 elements
- Elements are stored in a row-wise fashion
  - All the entries in the first row are stored first, followed by the entries in the second row, which are followed by the matrix in the third row, etc.
- We use `malloc` to request the 1D array
  - We'll have to use `free` once when we're done
- Executable called "`matrix1D.exe`"

Measure the Time of a Function in a C++ Program

- std::chrono is a C++ library for measuring time in your code (since C++11)
  - The wall clock is the normal time we are used to
  - A steady clock can be used **only to measure relative times**

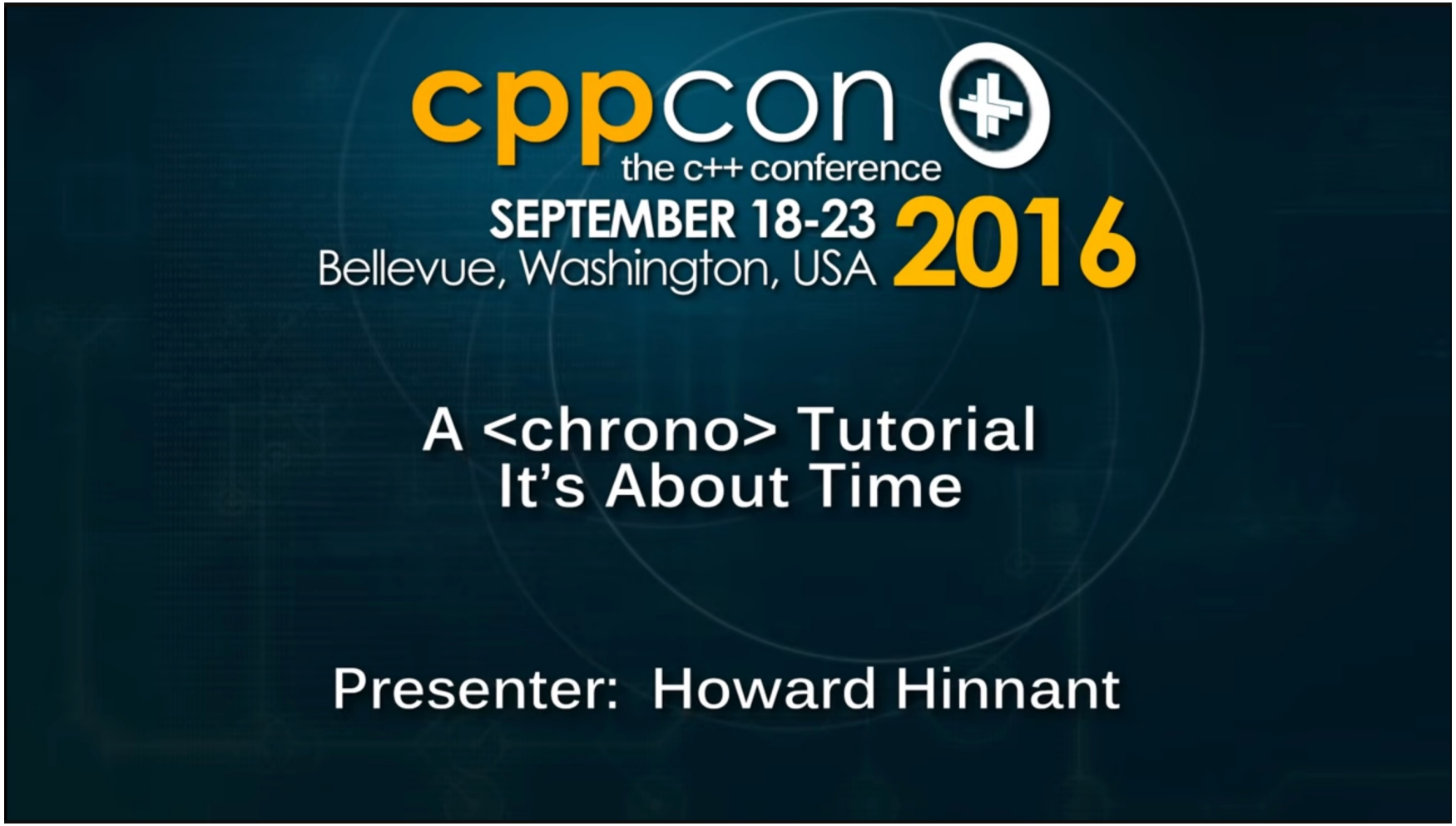
```
void long_function() {
    // Place your function code here
    for (int i = 0; i < 1000000; ++i) {
        // Some computation
    }
}

int main() {
    auto start_time = std::chrono::steady_clock::now();

    // Call the function whose runtime you want to measure
    long_function();

    auto end_time = std::chrono::steady_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end_time - start_time);

    std::cout << "Runtime: " << duration.count() << " microseconds" << std::endl;
}
```

Measure Time in C++ (std::chrono) – cont'd

- You can choose other clock types
  - Ex: std::chrono::high_resolution_clock represents the smallest tick period
  - Ex: std::chrono::system_clock represents the system-wide real time clock

```
void long_function() {
    // Place your function code here
    for (int i = 0; i < 1000000; ++i) {
        // Some computation
    }
}

int main() {
    auto start_time = std::chrono::high_resolution_clock::now();

    // Call the function whose runtime you want to measure
    long_function();

    auto end_time = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end_time - start_time);

    std::cout << "Runtime: " << duration.count() << " microseconds" << std::endl;
}
```

A C++ Chrono Tutorial



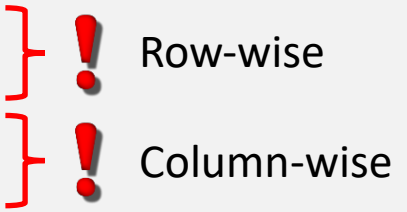
https://www.youtube.com/watch?v=P32hvk8b13M

Add Elements in the Matrix (via its Linearized 1D Array)

- Locality (row-wise/row-by-row/row-major) vs non-locality (column-wise)

```
#include <chrono> // need this for timing
#include <stdio.h>
#include <stdlib.h>

struct squashedMatrix {
  /// 2D matrix stored row-wise in a one dimensional array
  unsigned int height; /// number of rows in matrix
  unsigned int width;  /// number of columns in matrix
  double *pMatVals;
};

int main(int argc, char *argv[]) {
  if (argc != 2) {
    printf("Pass 1 for row-wise, pass 2 for column-wise sum.\n");
    return 1;
  }
  int option = atoi(argv[1]);

  struct squashedMatrix mat1D;

  mat1D.height = 2000; // perhaps you set this based on user input
  mat1D.width = 2000;  // perhaps you set this based on user input
  mat1D.pMatVals = (double *)malloc(mat1D.height * mat1D.width * sizeof(double));

  unsigned int i, j;
  for (i = 0; i < mat1D.height; i++)
    for (j = 0; j < mat1D.width; j++)
      mat1D.pMatVals[i * mat1D.width + j] = 1. / (i + j + 1.);
```

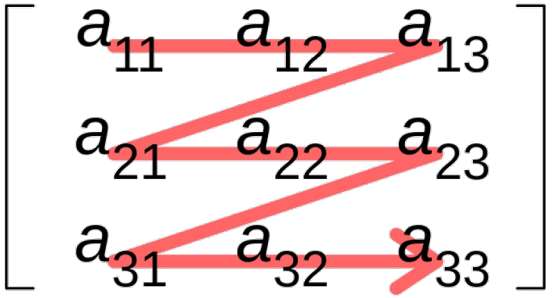
```
// Compute sum of elements
  double sum = 0.;
  __int64 ctr1 = 0, ctr2 = 0, freq = 0; // need this for timing
  auto beg = std::chrono::steady_clock::now();
    if (option == 1) {
      for (i = 0; i < mat1D.height; i++)
        for (j = 0; j < mat1D.width; j++)
          sum += mat1D.pMatVals[i * mat1D.width + j];
    } else if (option == 2) {
      for (j = 0; j < mat1D.width; j++)
        for (i = 0; i < mat1D.height; i++)
          sum += mat1D.pMatVals[i * mat1D.width + j];
    } else {
      printf("Bad input option.\n");
      return 1;
    }
  auto end = std::chrono::steady_clock::now();
  auto sec = std::chrono::duration_cast<std::chrono::microseconds>(end-beg).count();


  printf("The sum of the elements is: %f\n", sum);
  printf("Time spent in microseconds: %f\n", sec);

  // Done with the matrix; free the mem
  free(mat1D.pMatVals);

  return 0;
}
```



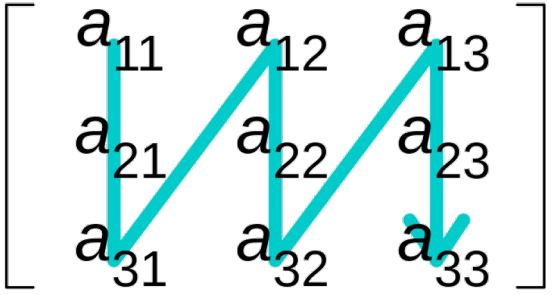
Speed of execution: 6 to 7 times faster if accessed w/ locality

```
TRUMAN Release> ./matrixRep1D.exe
Pass 1 for row-wise, pass 2 for column-wise sum.
TRUMAN Release>
TRUMAN Release>
TRUMAN Release> ./matrixRep1D.exe 1
The sum of the elements is: 2772.088785
Time spent in microseconds: 0.003622
TRUMAN Release>
TRUMAN Release>
TRUMAN Release> ./matrixRep1D.exe 2
The sum of the elements is: 2772.088785
Time spent in microseconds: 0.023060
TRUMAN Release>
```

Row-major order



Column-major order



OS Name	Microsoft Windows 10 Enterprise 2016 LTSB
System Type	x64-based PC
Processor	Intel(R) Core(TM) i7-6800K CPU @ 3.40GHz, 3401 Mhz, 6 Core(s), 12 Logical Processor(s)
Installed Physical Memory (RAM)	32.0 GB
Compiler	Visual Studio 2015 (Enterprise)

Taking it to the next level: 3D Matrices – good locality

```
// sum up all entries in 3D matrix with locality
for (int i = 0; i < M; i++)
    for (int j = 0; j < M; j++)
        for (int k = 0; k < M; k++)
            sum += bPPP[i][j][k];
```

Execution time: 0.308178 microseconds

OS Name	Microsoft Windows 10 Enterprise 2016 LTSB
System Type	x64-based PC
Processor	Intel(R) Core(TM) i7-6800K CPU @ 3.40GHz, 3401 Mhz, 6 Core(s), 12 Logical Processor(s)
Installed Physical Memory (RAM)	32.0 GB
Compiler	Visual Studio 2015 (Enterprise)

Taking it to the next level: 3D Matrices – bad locality

```
// sum up all entries in 3D array
for (int k = 0; k < M; k++)
    for (int j = 0; j < M; j++)
        for (int i = 0; i < M; i++)
            sum += bPPP[i][j][k];
```

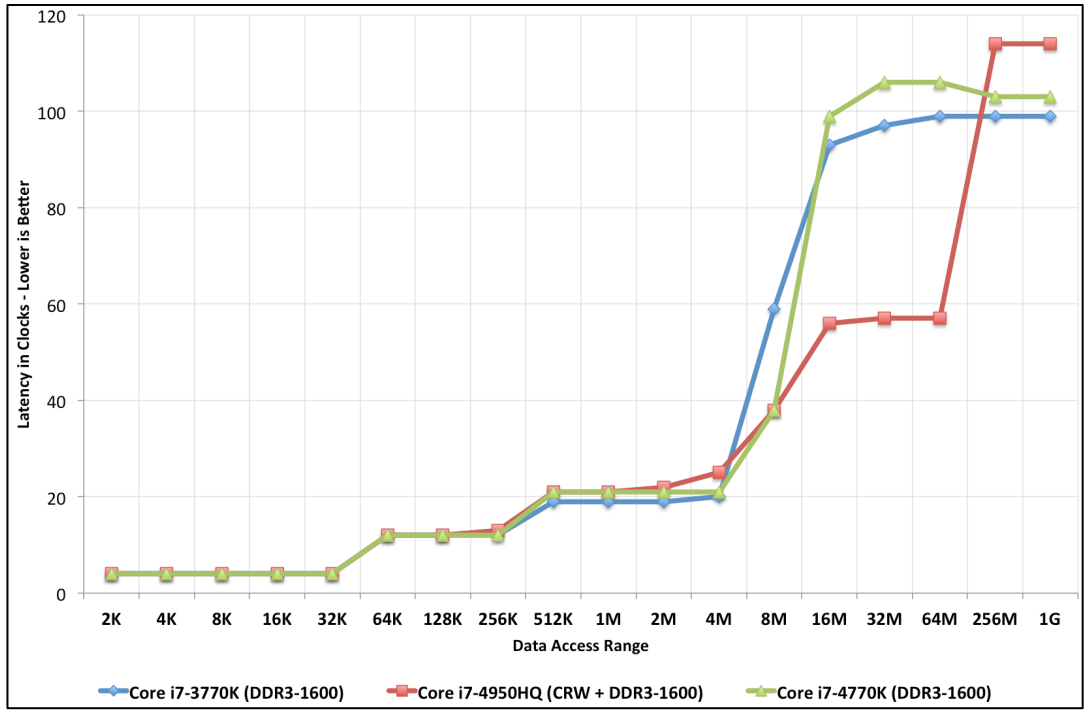
Execution time: 9.33842 microseconds (30x slower)

OS Name	Microsoft Windows 10 Enterprise 2016 LTSB
System Type	x64-based PC
Processor	Intel(R) Core(TM) i7-6800K CPU @ 3.40GHz, 3401 Mhz, 6 Core(s), 12 Logical Processor(s)
Installed Physical Memory (RAM)	32.0 GB
Compiler	Visual Studio 2015 (Enterprise)

The Locality-aware Implementation is 30x Faster

- First time we discuss in the class how to improve performance
- Different access patterns can result in different hardware efficiency and memory access latency
- Why? Locality-aware implementation allows us to reduce the number of trips between processor memory (cache) and main memory
  - Each data access brings more than the size of that data into the so-called cache line (64 bytes)
    - L1 cache: 4 cycles
    - System memory access: 100-120 cycles

Latency vs Data Access Range



Message in the plot above: The longer the trip, the higher the access latency thus slower runtime.

Quiz: Qualitative Estimates of Locality

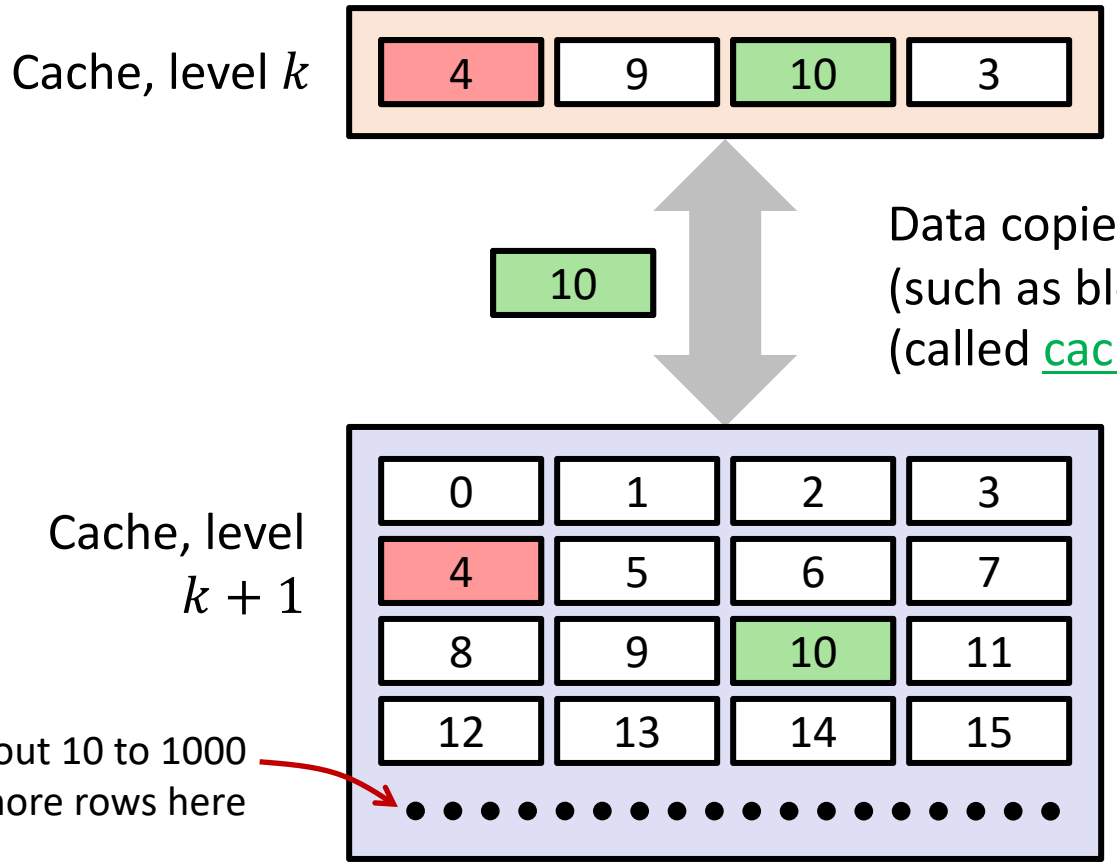
- Question: Does this function have good locality with respect to array a?

```
int sum_array_cols(int* a, int N, int M)
{
        int i, j, sum = 0;

        for (j = 0; j < N; j++)
            for (i = 0; i < M; i++)
                sum += a[i * N + j];
        return sum;
}
```

- Note: Being able to look at code and get a qualitative sense of its locality is a key skill for a professional programmer

Organization of Cache

Everything we discuss here is handled in hardware. Invisible to programmer



Smaller, faster, more expensive memory can store a subset of the main memory blocks

Data copied in blocks of fixed-size transfer units. Each block (such as block 4, for instance) typically holds 64 bytes of data (called cache line) – the smallest unit of data movement

Larger, slower, cheaper memory (shown here as partitioned into "blocks" that are mapped into cache lines)

Cache Line (aka Cache Block)

- When data is brought in cache because a piece of data is needed, the memory management unit (MMU) brings more than just the needed piece of data
  - Ex: a = b + 1; (application only needs two integers (8 bytes), a and b, but MMU will bring 64 bytes
  - Why? Because overhead is not high for bringing more data since we have wide buses; that is, the cost in bringing more data incurs very little extra cost
  - By bringing more, we can save the time for future data access (i.e., no need to bring again)

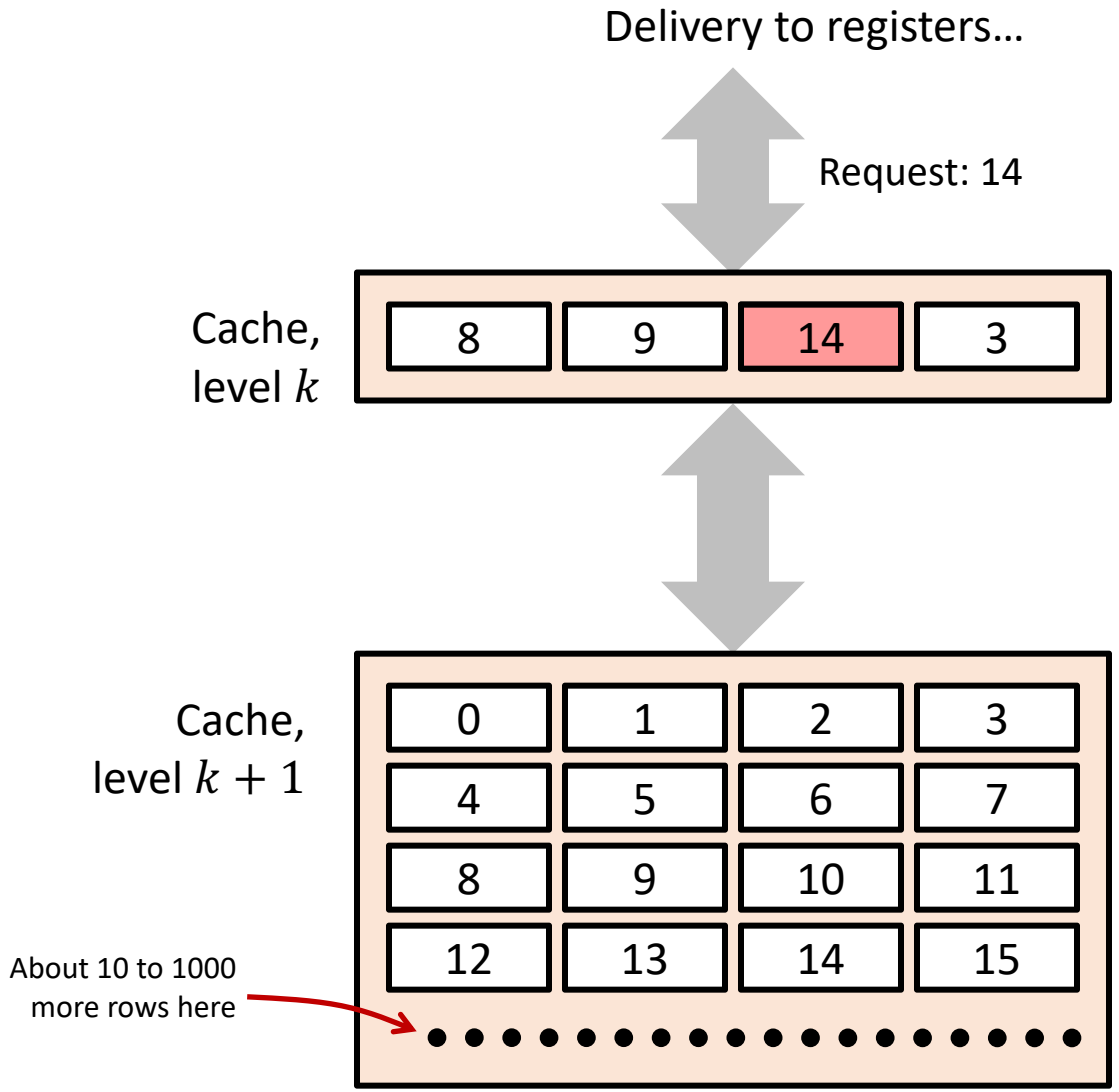
- Consequence: you will reduce the number of trips to main memory if you use multiple times data from a cache line

- "cache line" – the minimum data size per fetch by CPU
  - Many systems have cache line of 64 bytes (typically the same for L1, L2, and L3 caches)
  - C++ abstract this concept through std::hardware_destructive_interface_size

From Programmer's Perspective

- We want to write a program that can maximally utilize the advantage of cache line
- How? Pay attention to locality when you write your programs
  - Temporal locality: accessed data refers to the same location (the same cache line)
  - Spatial locality: accessed data are close to each other (the same cache line or adjacent cache lines)
- Takeaway: While we don't directly program cache line and data movement, a program with good locality pattern allows the compiler to optimize your code a lot easier than that without locality
  - Advantage is quantified by how many times you *hit* the cache and *miss* the cache when accessing data

General Cache Concepts: Hit



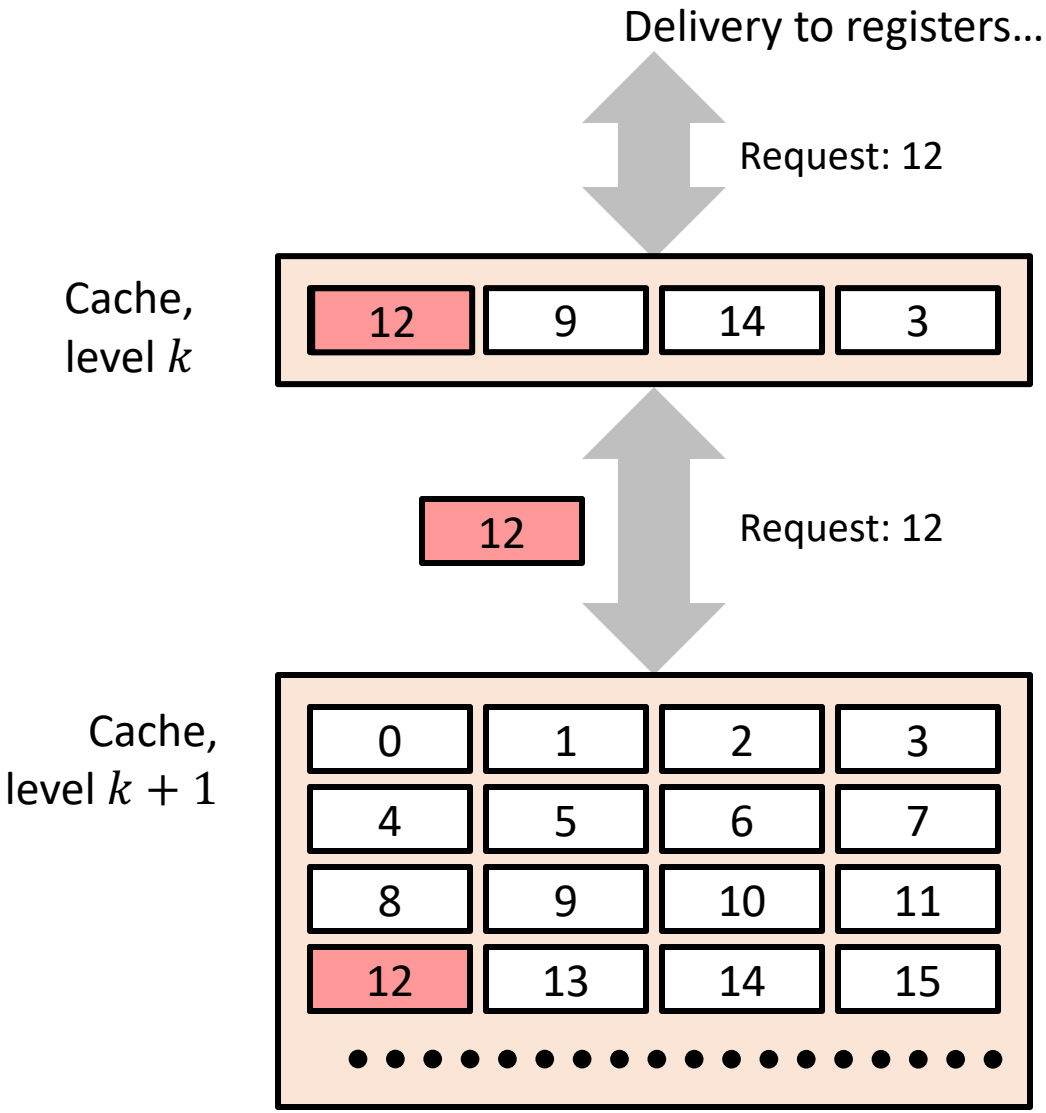
Data in block 14 is needed

Block 14 is in cache: *Hit!*

Hit Rate: on average, how many out of 100 mem requests hit the cache?

The higher the hit rate, the better the performance (usually).

General Cache Concepts: Miss



Data in block 12 is needed

Block 12 is not in cache:
Miss!

Block 12 is fetched from *the next memory hierarchy*

Block 12 is stored in cache

- Placement policy: determines where block 12 should go
- Replacement policy: determines which block gets evicted (victim) so 12 can be stored.

Most common reasons for Cache Misses

- Cold (compulsory) miss
  - Happens when cache is empty (e.g., beginning of the program)
- Capacity miss
  - Happens when: not enough space in cache, data working set is too large
  - Example: the active part of main memory overwhelms in size what can be accommodated by the cache
    - You are asking for more than what level k can accommodate (fit)
- Conflict miss
  - Happens when: Enough space, but you have bad luck (lots of "conflicts", as dictated by replacement policy)
  - Fact: In the memory hierarchy, the cache blocks at level $k+1$ map into a small subset of block positions at level k
    - Example (see previous slide): block i at level $k+1$ must be placed in block (i % 4) at level k
  - Conflict misses occur when the level k cache is large enough, but multiple data objects all map to the same block in the level k cache.
    - Example (see previous slide): Referencing in this order blocks 0, 8, 0, 8, 0, 8, ... would miss every time (since $0 \,\%\, 4 = 8 \,\%\, 4$)

When Cache Miss Occurs: Placement & Replacement Policies

- Concept: when somebody comes in, somebody else has to be evicted

- **Caching: like bringing visitors from a large city to live in one apartment building next to some vista points**
  - Vista points: the registers and the ALU/CU
  - "Placement policy": decide at which "floor of the building" to host the new visitors (new visitors ≡ new cache line)

- If there are 2 or 4, etc. apartments (cache lines) at each floor, which one do you choose?
  - "Replacement policy" – if you have *Option A* and *Option B*, which one should you choose: A or B?
    - Will you kick out tenants from apartment A, or tenants from apartment B?

- This is a big topic in computer architecture – briefly outlined in the next few slides (bonus)

Placement & Replacement Policies [continued]

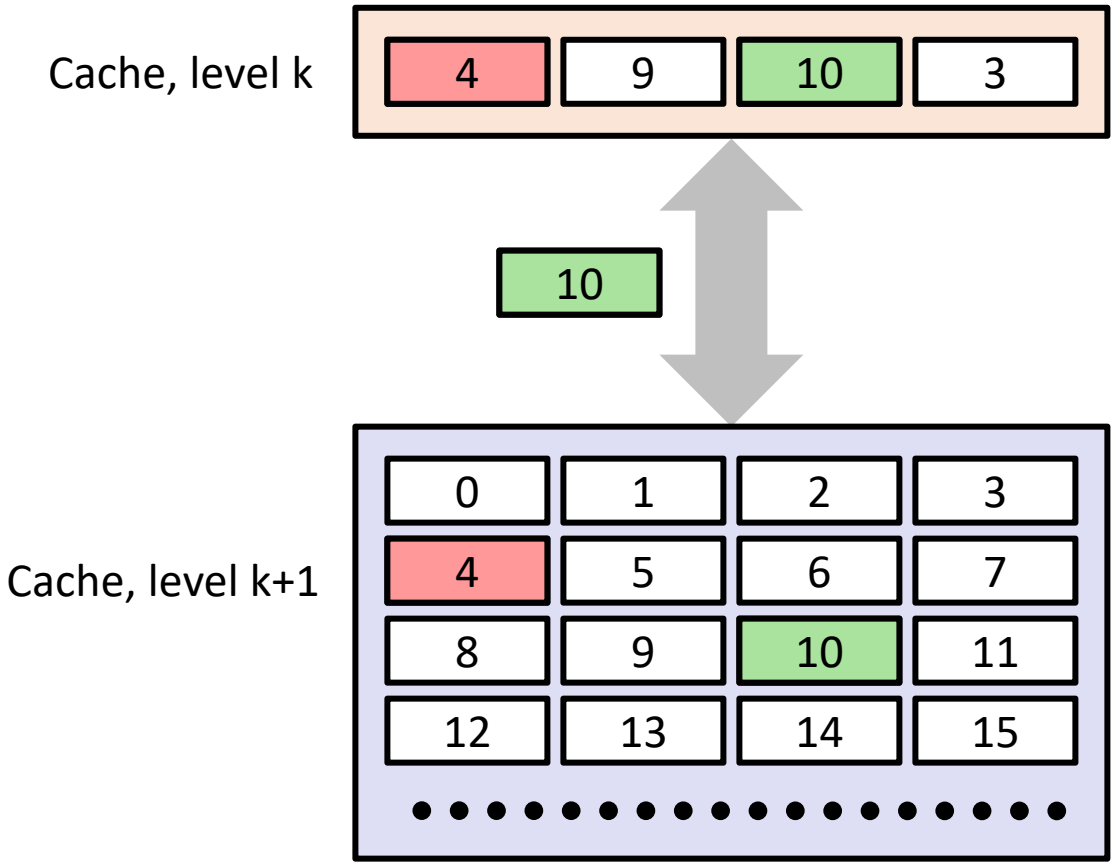
- **Direct-mapping cache:** cache structured as skinny skyscraper, with one apartment per story
  - An "apartment" large enough to hold 64 folks (Bytes)
  - One cache line per story
  - Map each block of main memory to only one specific cache location

- **K-way associative cache:** a certain number of stories, each with K apartments
  - K cache lines per story
  - Adopted by conventional computer architectures

- **Fully associative cache:** apartment complex with ground level only (ranch)

Placement & Replacement Policies: further discussion

- Direct map (the tall skinny skyscraper)
  - Pros: very fast to find if in cache or not
  - Cons: sometimes, you'll have to evict a cache line that sees a lot of use (no freedom at all in the placement policy)



Placement & Replacement Policies: further discussion

- Fully associative (like the ranch)
  - Pros: allows to store an address into any entry.
  - Cons:
    - In order to check if a particular address is in the cache, it has to compare all current entries

Placement & Replacement Policies: further discussion

- K-way associative (like Hilton Garden Inn)
  - Decent compromise:
    - You find the floor of your room first and then search the room on that floor
    - You don't have to search too many "apartments" at a floor level for a matching room
    - It yields a reasonably decent replacement policy – you have K options to choose from
  - NOTE: Intel and AMD chips - 2, 4, 8, 16-way associative
    - Depends on cache level and which chip model

[New Topic:] General Cache Organization

[Symbols Used and Their Meaning]

- B: Number of bytes in a cache line
  - Typically, B is 64 bytes
  - Or, abstracted in C++ std::hardware_destructive_interface_size
- E: The number of cache lines ("Elements") that bundle together (apartments on a floor level)
  - Such a huddle is called "a set of cache lines"
  - Typically, E is 1, 2, 4, 8, or 16 (other values possible)
- S: the number of "Sets" that make up the cache (number of stories/floors in the building)
- T: total cache size in bytes: $B \times E \times S = T$

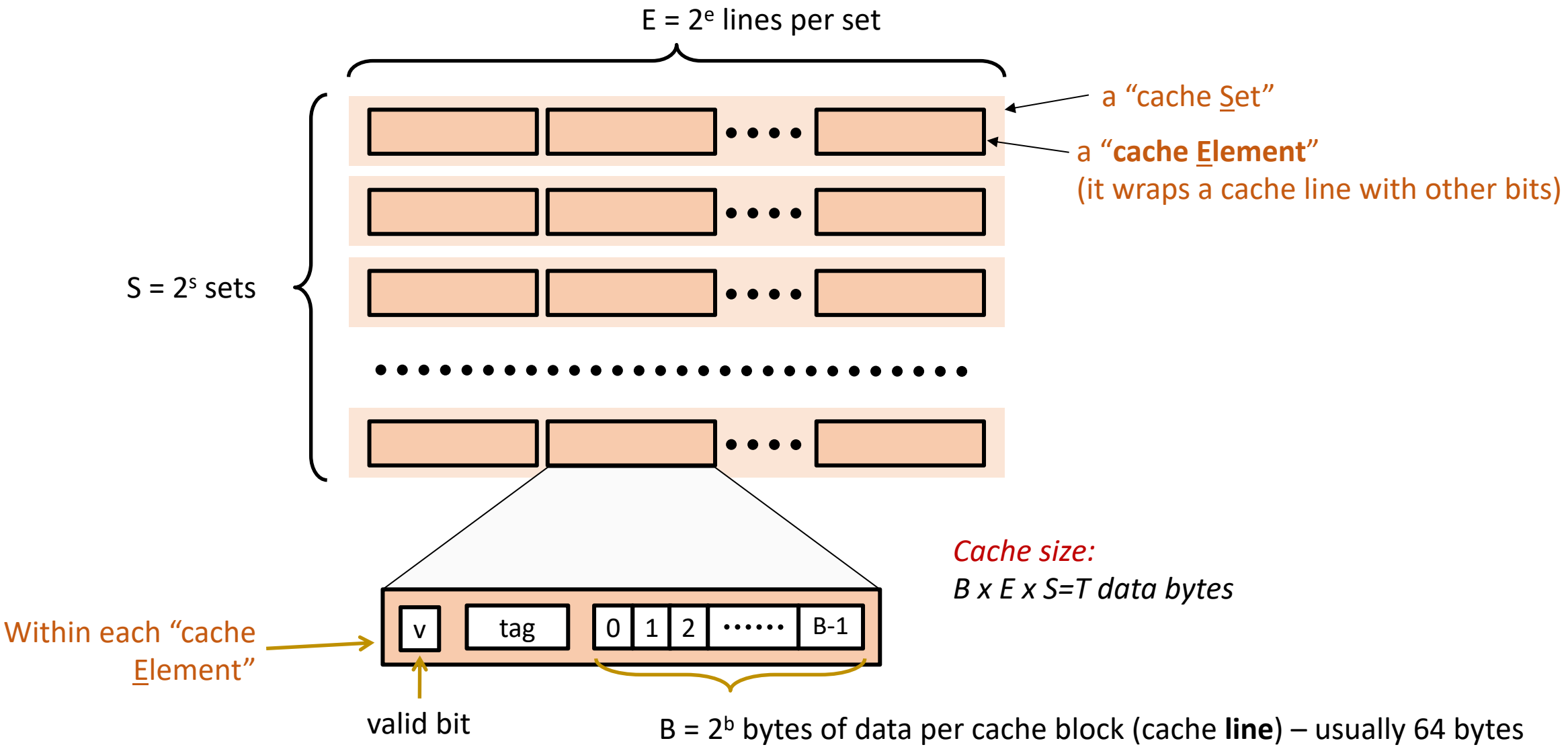
The BEST Rule under Our Apartment Building Metaphor

- Visitors from big city driven to an apartment building to be in the proximity of big event
  - How many folks will fit in this building?

- The total number T of folks in the building depends on
  - How many folks in an apartment (B)
  - How many apartment per floor level (E)
  - How many stories/floors in the building (S)

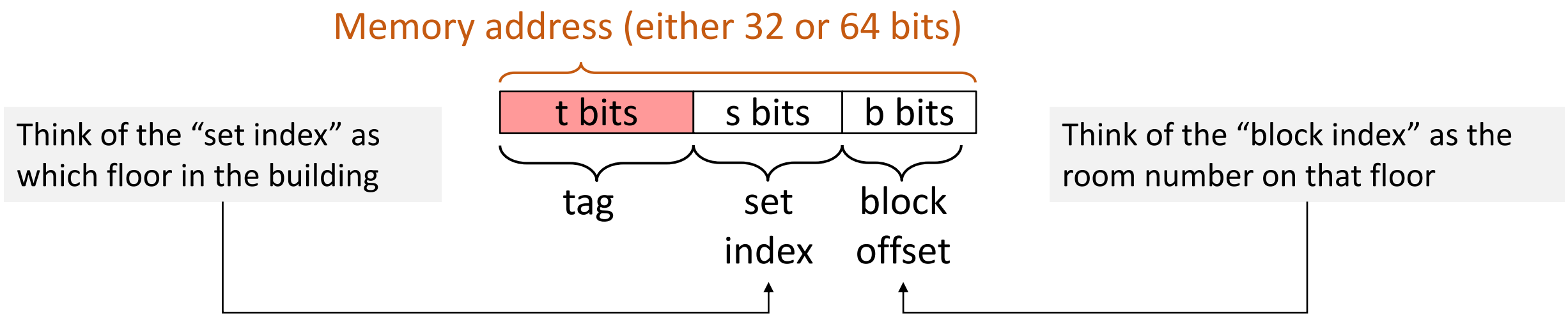
- Total number of folks in the building: $\mathrm{B} \times \mathrm{E} \times \mathrm{S} = \mathrm{T}$

General Cache Organization (B, E, S)

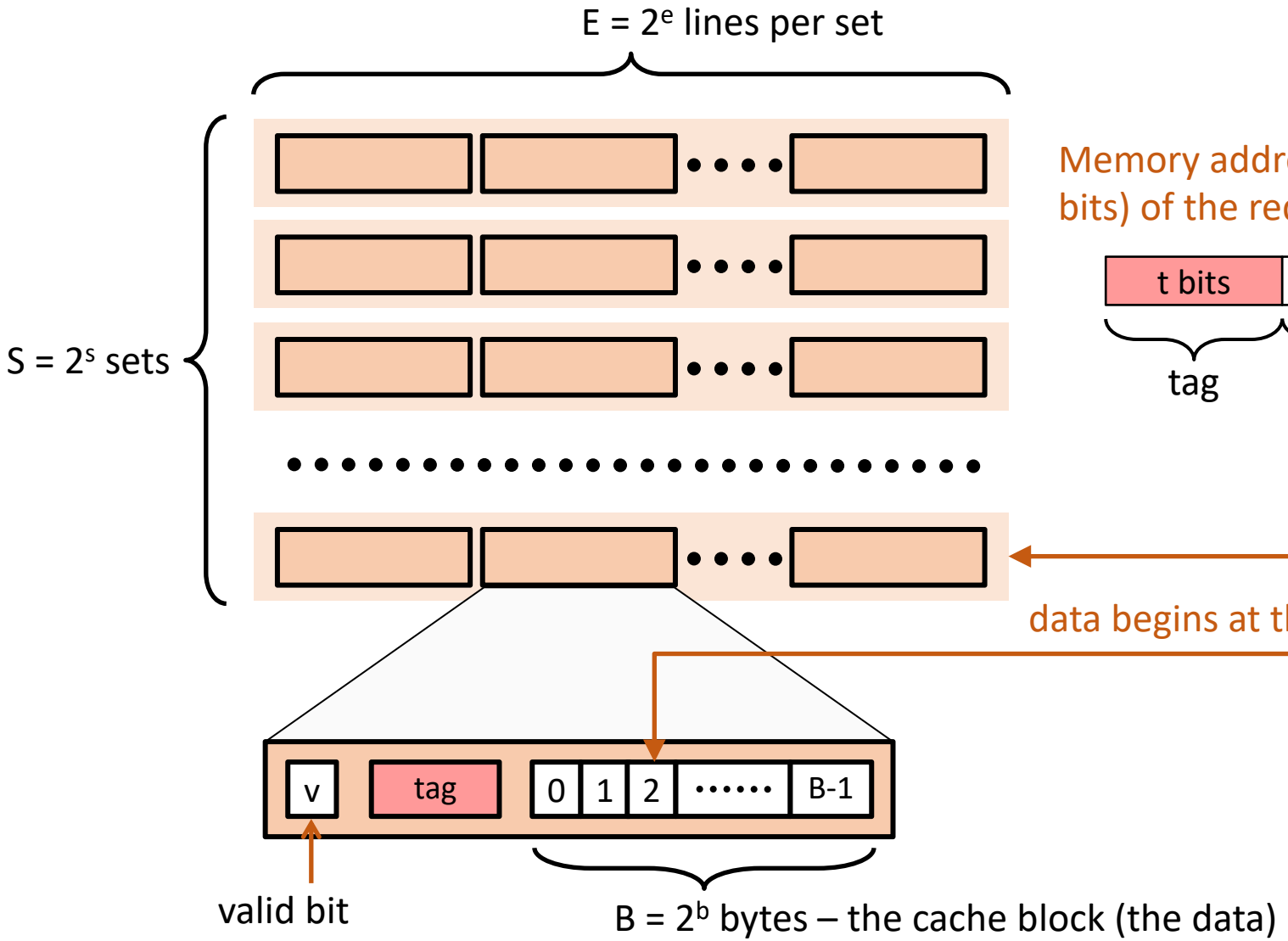


On finding if data from a memory address is in cache or not

- The mission: Bring (read) into a register a variable at the given memory address
- The key question: Is that variable in cache?
- To answer this question, the address information is partitioned into three chunks



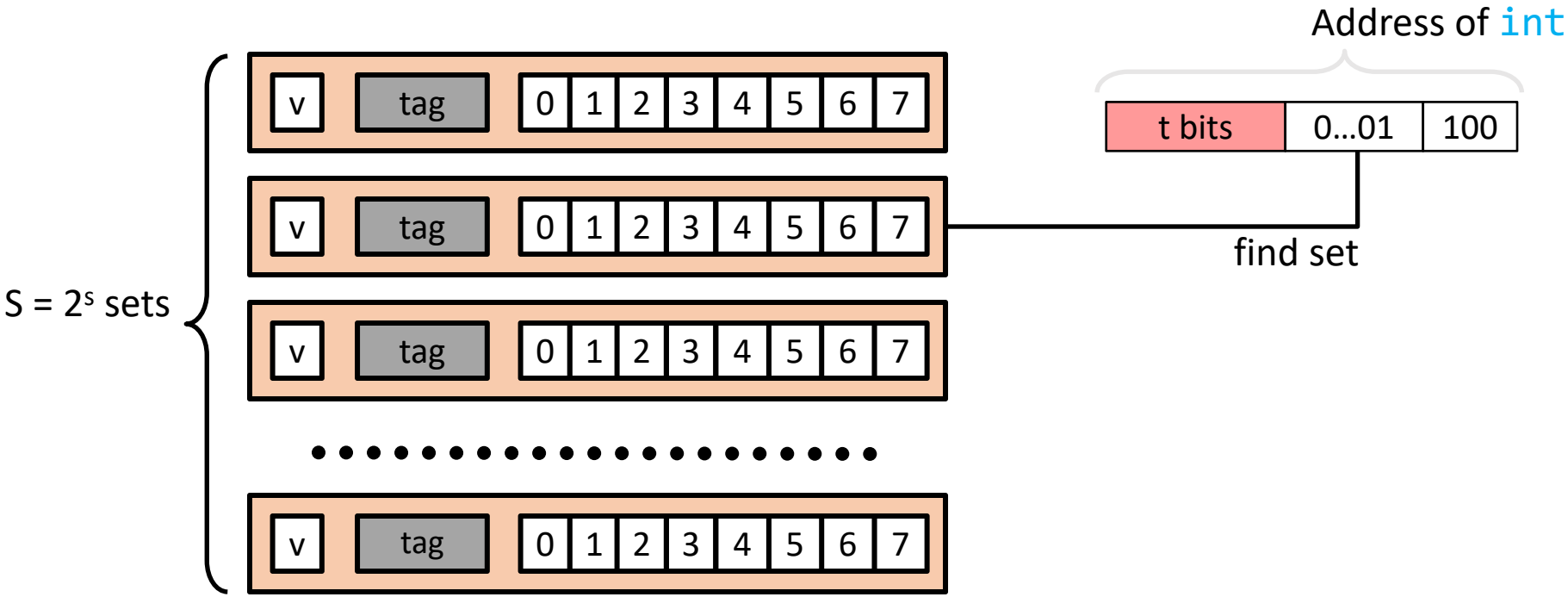
How Cache Read Takes Place



- ***First thing**: Locate set (floor)*
- *Check if any element in set has a matching tag*
- *Yes + line valid = hit*
  - *Locate data starting at offset*

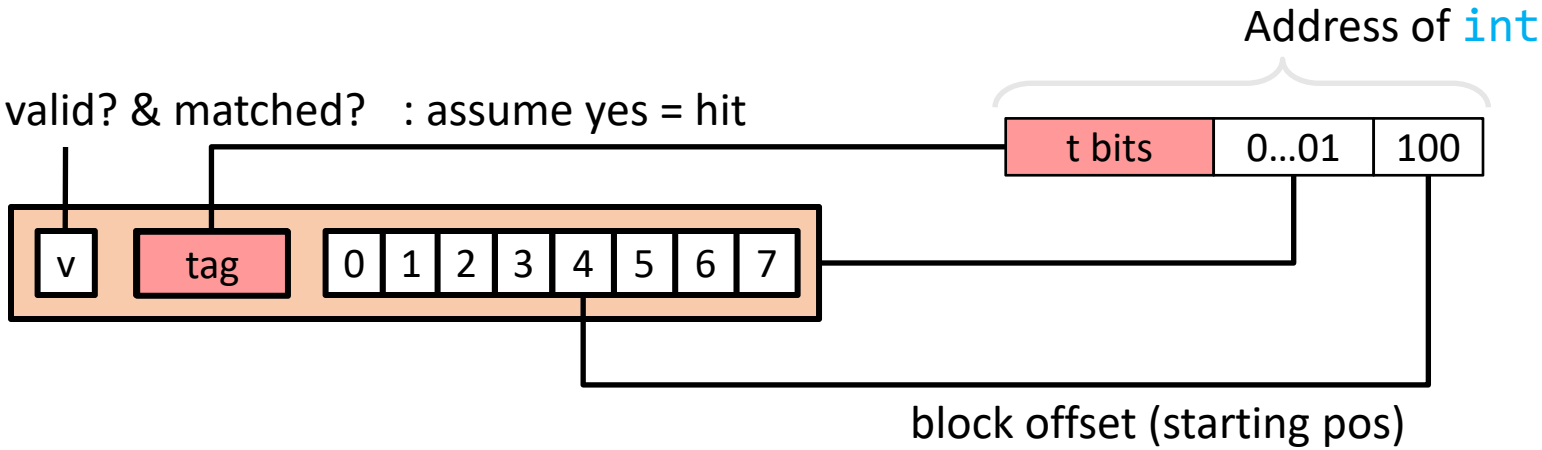
Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set (special case of k-way mapping with k=1)
Assume: cache block size 8 bytes (instead of 64 for brevity)



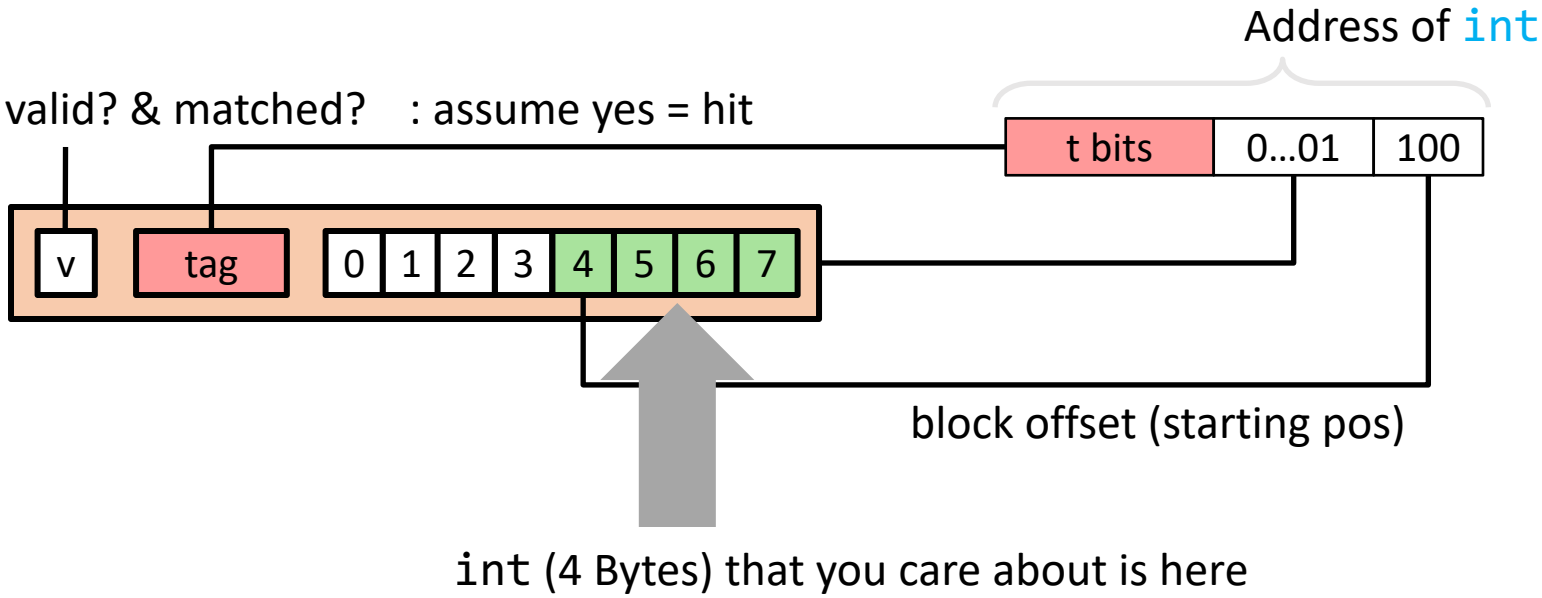
Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set (special case of k-way mapping with k=1)
Assume: cache block size 8 bytes (instead of 64 for brevity)



Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set (special case of k-way mapping with k=1)
Assume: cache block size 8 bytes (instead of 64 for brevity)

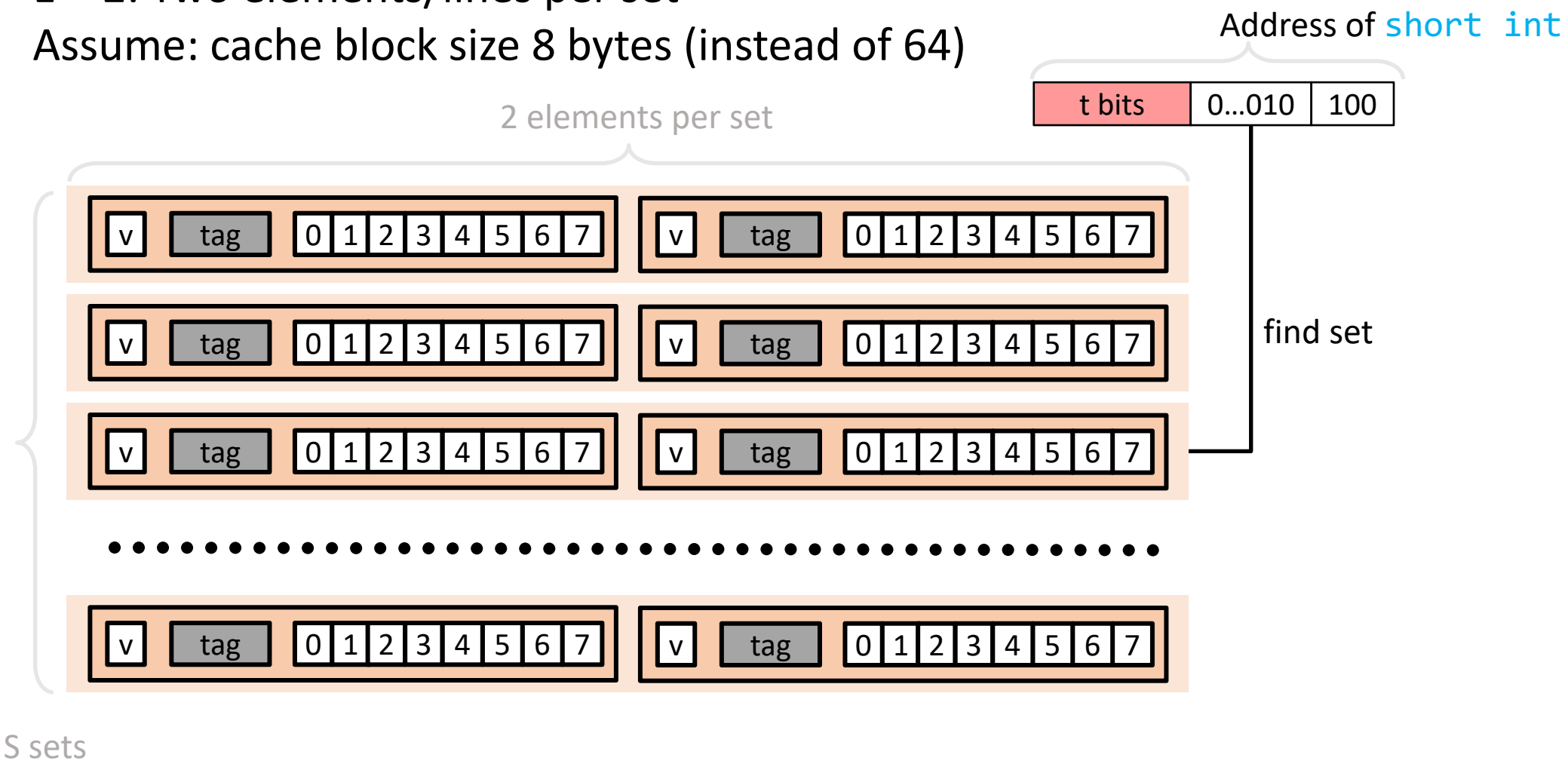


If tag doesn't match: old line is evicted and replaced

E-way Set Associative Cache (Here: E = 2)

E = 2: Two elements/lines per set

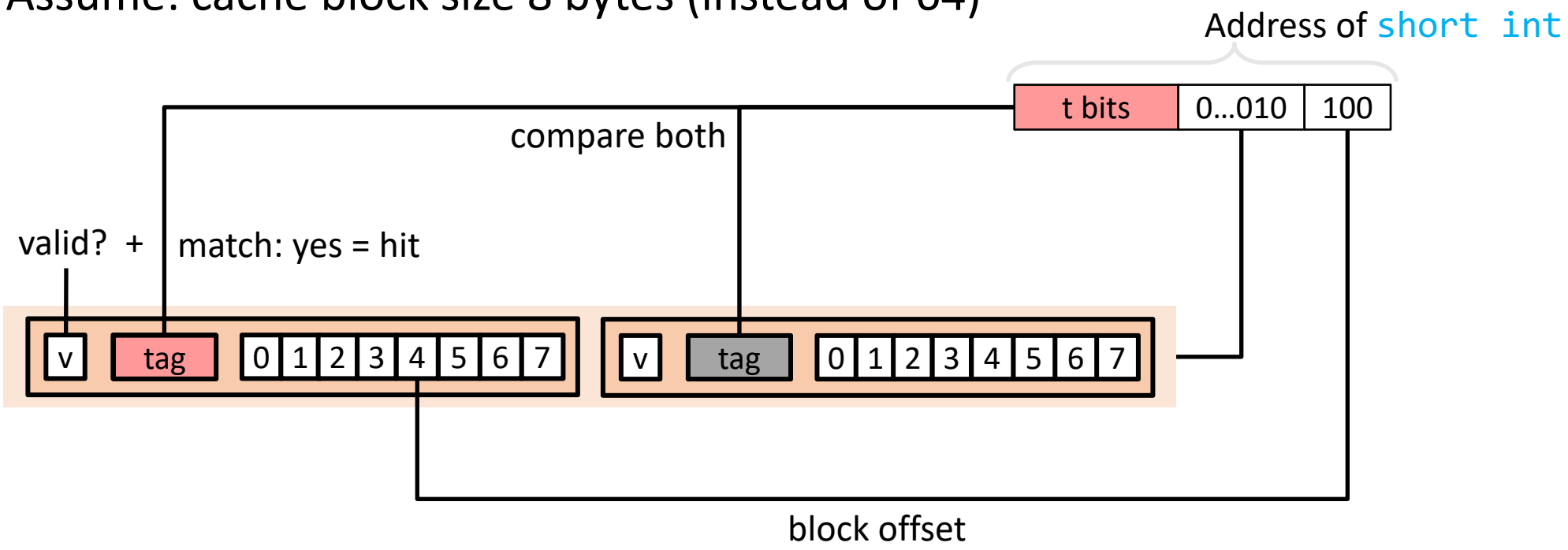
Assume: cache block size 8 bytes (instead of 64)



E-way Set Associative Cache (Here: E = 2)

E = 2: Two elements/lines per set

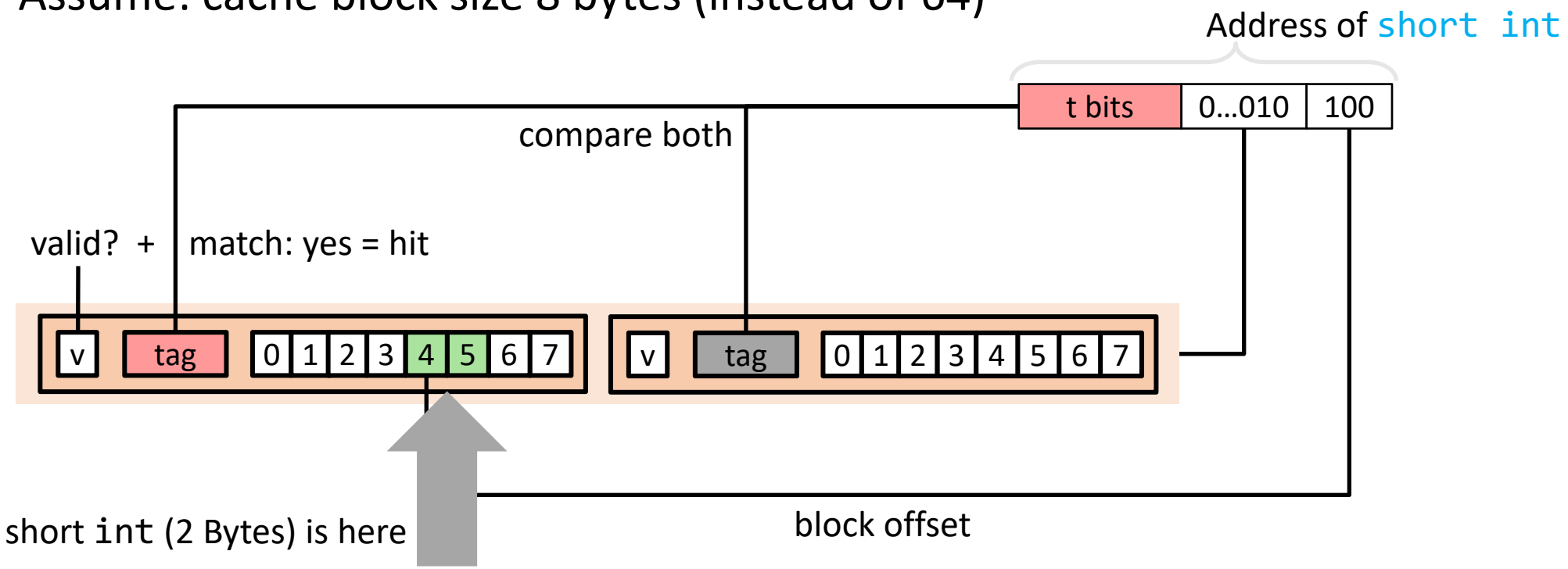
Assume: cache block size 8 bytes (instead of 64)



E-way Set Associative Cache (Here: E = 2)

E = 2: Two elements/lines per set

Assume: cache block size 8 bytes (instead of 64)



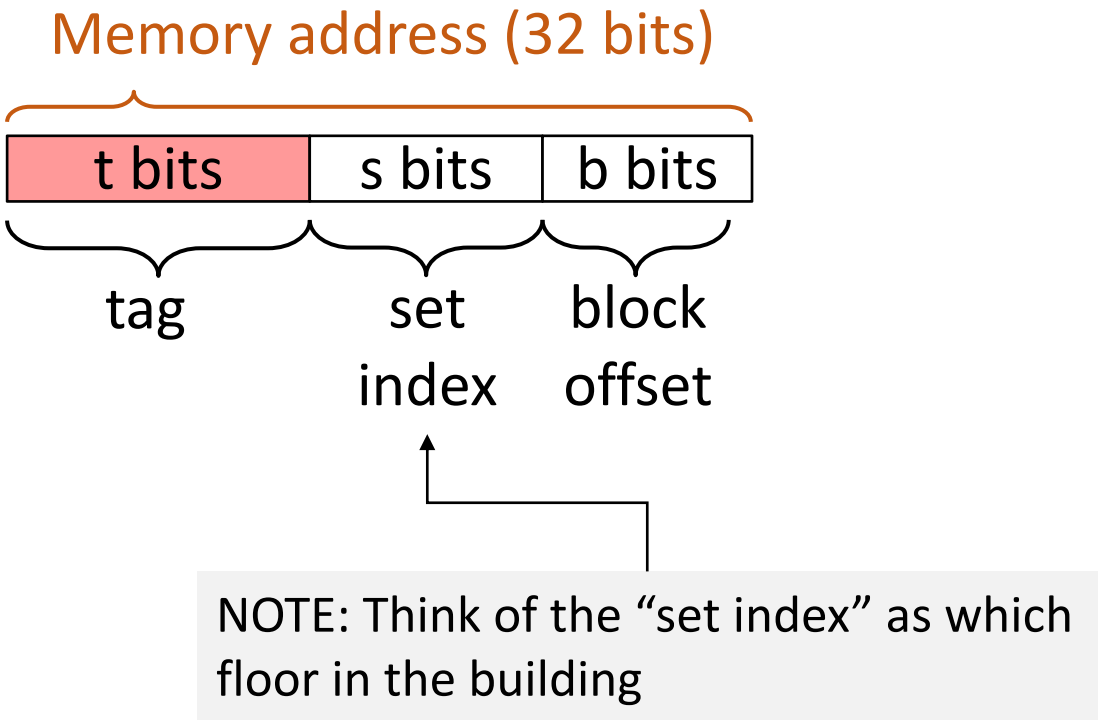
If nothing matches:

- One line in set is selected for eviction and replacement
- Replacement policies: random, least recently used (LRU), …

Quiz: Direct-mapping Cache

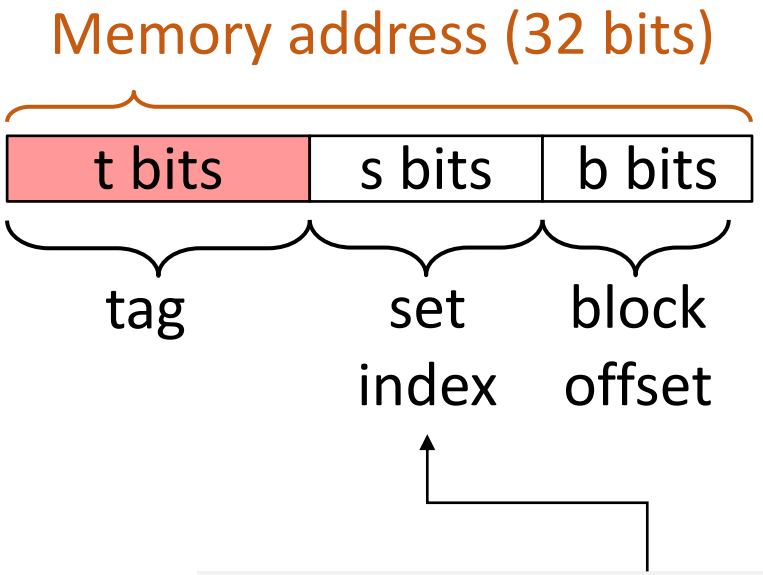
- Assume that
  - Address is on 32 bits
  - We have a direct mapping setup
  - Cache line is 64 bytes
  - Cache size is 32 KBytes (2^{15} bytes in total)
    - Note: 1 KByte is 2^{10} Bytes

- What is the value of B?
- What is the value of E?
- What is the value of S?
- How many bits in the tag?



Quiz: Direct-mapping Cache

- Assume that
  - Address is on 32 bits
  - We have a direct mapping setup
  - Cache line is 64 bytes
  - Cache size is 32 KBytes (2^{15} bytes in total)
    - Note: 1 KByte is 2^{10} Bytes

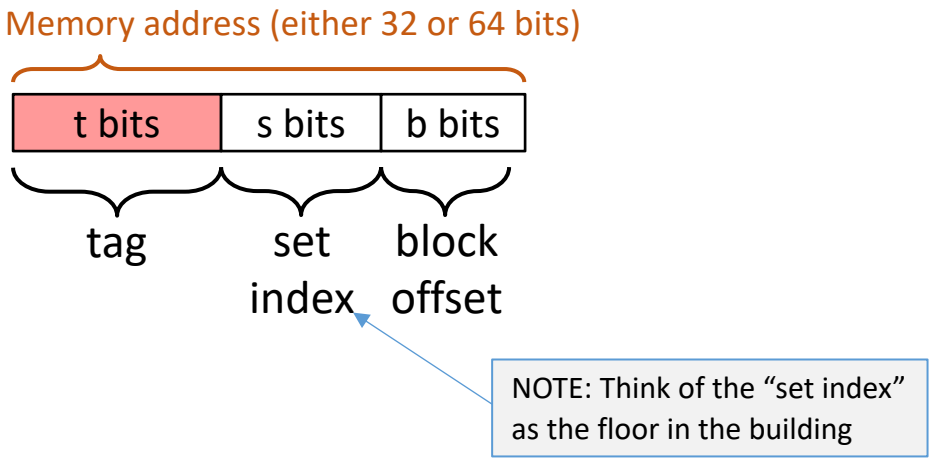


NOTE: Think of the "set index" as which floor in the building

- What is the value of B? 64
- What is the value of E? 1
- What is the value of S? $2^{15} / 2^{6} = 2^{9}$ (s=9)
- How many bits in the tag? 32 – 9 – 6 = 17

Quiz: How large can the set index be for a fully associative mapping?

- What is the value of s for a fully associative mapping?
- What is a disadvantage of this?
- What is an advantage of this?



Takeaway: How Cache is Related to Performance?

- **Cache lines are the Unit of Memory Transfer**
  - CPUs **do not fetch individual bytes** but instead **load entire cachelines** (typically **64 bytes**) from RAM to cache – why? Bringing more data incurs very little extra cost due to hardware capacity (bus)
  - **Reading adjacent data** is much faster because it's likely already in the cache.

- **Spatial Locality: Access Nearby Data Efficiently**
  - If a program accesses memory at address X, it's likely to access nearby addresses (X+1, X+2, etc.).

- **Temporal Locality: Reuse Data Before It's Evicted**
  - If a program accesses memory at address X, it's likely to access X again soon.

- **Row-Major vs. Column-Major: Cache Matters!**
  - **Row-major (C, Python)**: Accessing elements **row-wise** is **cache-friendly**
  - **Column-major (Fortran, MATLAB)**: Accessing **columns** is more efficient in **column-major storage**